

ECS 关系 (ECS Relationships)

1. ECS 关系介绍

在构建 Bevy 应用时，将实体“关联”在一起通常是很有用的。其中最常见的情况是将父实体和子实体连接起来。ECS 关系就是用来满足这个需求的一个通用且高效的、基于组件的系统，用于双向地链接实体。

2. 在拥有 ECS 关系之前

在以前的 Bevy 版本中，子实体会拥有一个 Parent 组件，用于存储其父实体的引用，而父实体则会拥有一个 Children 组件，用于存储其所有子实体的列表。为了确保这些连接的有效性，开发者不能直接修改这些组件，而必须通过专用的命令进行更改。

这种做法虽然有效，但也存在一些明显的缺点：

- 层级关系的维护是“独立于”核心 ECS 数据模型的。这使得改进生成 (spawn) API 变得困难，并且与层级结构的交互不够自然。
- 系统是专用的，难以复用。开发者若想定义自己的关系类型，必须重新造轮子。
- 为了确保数据完整性，需要进行代价高昂的扫描以避免重复数据。

3. 创建 ECS 关系

```
#[derive(Component)]
#[relationship(relationship_target = LikedBy)]
pub struct Likes(pub Entity);    // 不可变组件，事实来源

#[derive(Component)]
#[relationship_target(relationship = Likes)]
pub struct LikedBy(Vec<Entity>);    // 可变组件，Vec 的访问权限必须为私有

fn setup(mut commands: Commands) {
    // (Name, LikedBy)
    let e0 = commands.spawn(Name::new("0")).id();
    // (Likes, LikedBy, Name)
    let e1 = commands.spawn((Likes(e0), Name::new("0-0"))).id();
    // (Likes, Name)
    commands.spawn((Likes(e0), Name::new("0-1")));
    // (Likes, Name)
    commands.spawn((Likes(e1), Name::new("0-0-0")));

    // 0      0-0      0-0-0
    //      0-1
}
```

4. 事实来源

Relationship 组件是“事实来源” (source of truth)，而 RelationshipTarget 组件则会根据它进行更新。这意味着添加或移除关系必须始终通过 Relationship 组件来完成。

我们采用这种“事实来源”模型而不是允许两个组件共同驱动，主要是出于性能考虑。如果允许双方都能写入，就必须在插入时进行昂贵的扫描，以确保两者保持同步且无重复。而以 Relationship 为唯一数据源的做法，使得添加关系成为常数时间操作（这比旧的 Bevy 父子结构更高效!）。

5. 保证数据完整性

关系系统建立在 Bevy 的组件钩子 (Component Hooks) 之上，能够立即且高效地维护 Relationship 与 RelationshipTarget 之间的连接，通过直接插入组件的添加/移除/更新生命周期实现这一点。再结合新的不可变组件 (Immutable Components) 功能 (关系组件是不可变的)，这保证了无论开发者做什么，数据完整性始终得以保持！

6. 新父子级

Bevy 现有的层级系统已被新的 ChildOf Relationship 和 Children RelationshipTarget 完全取代。添加一个子实体现在非常简单：

```
commands.spawn(ChildOf(some_parent));
```

同样地，重新设定实体的父级 (reparenting) 也很简单：

```
commands.entity(some_entity).insert(ChildOf(new_parent));
```

7. ECS 关系的未来

请注意，这只是关系系统的第一步。我们还有扩展计划：

- 多对多关系 (Many-To-Many Relationships)：当前系统为一对多 (例如：ChildOf 关系指向一个目标实体，而 Children 目标组件可以被多个子实体指向)。有些关系类型可能受益于支持多个关系目标。
- 关系碎片化 (Fragmenting Relationships)：当前系统中，关系组件会像普通组件一样根据其类型对 ECS 原型 (archetypes) 进行碎片化 (例如：(Player, ChildOf(e1)) 和 (Player, ChildOf(e2)) 存在于同一个原型中)。“关系碎片化”将是一个可选机制，根据组件的值进一步分片，从而使具有相同关系目标的实体被存储在一起。这类似于索引，使得按值查询更快，并让某些访问模式更适合缓存。